

Universidad Torcuato Di Tella

Métodos Analíticos y Programación

Prof. Hernán Czemerinski

Clase teórica 1: Tipos de datos y expresiones

Datos

En general, un **programa** recibe **datos de entrada**, los procesa y genera **datos de salida**.

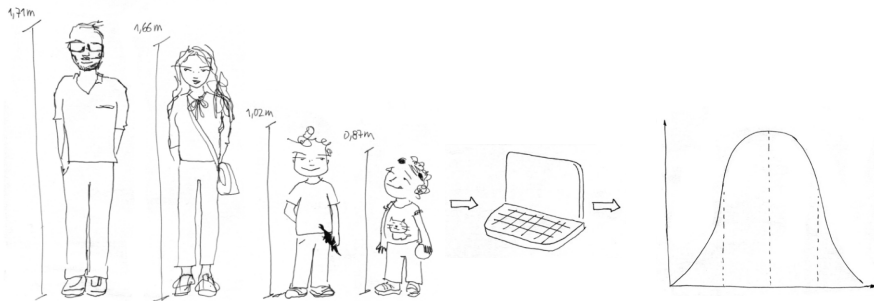


Ilustración: Klinko

Datos

En general, un **programa** recibe **datos de entrada**, los procesa y genera **datos de salida**.

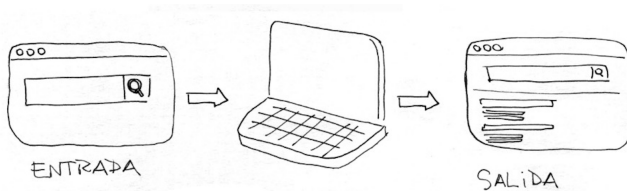


Ilustración: Klinko

Datos

En general, un **programa** recibe **datos de entrada**, los procesa y genera **datos de salida**.

Vamos a empezar por entender qué son los **datos** que un programa puede manejar. **A partir de la próxima clase, veremos qué hacer con ellos.**

Hay 4 tipos de datos básicos:

- ▶ Números enteros
- ▶ Números de punto flotante (o coma flotante)
- ▶ Caracteres
- ▶ Booleanos (verdadero/falso)

También existen los tipos compuestos: cadenas de caracteres, listas, conjuntos, diccionarios, etc. Y es posible **definir nuestros propios tipos**, para modelar los datos del problema a resolver.

Hoy vamos a ver algunos de estos tipos de datos. El resto, más adelante.

Números enteros (int)

Los **enteros** (int) para una computadora son similares a los enteros matemáticos (el conjunto $\mathbb{Z}$):

$$\dots, -4, -3, -2, -1, 0, 1, 2, 3, 4, \dots$$

con la *pequeña* salvedad de que **no son infinitos**: están acotados inferior y superiormente, según cuántos bits se usen para representarlos (típicamente, 32 o 64 bits). Esto depende del lenguaje y de la plataforma.

Ejemplo: con 4 bits se pueden representar 16 números diferentes:

$$\begin{array}{|c|c|c|c|} \hline 0 & 0 & 0 & 0 \\ \hline \end{array} \rightsquigarrow 0$$

$$\begin{array}{|c|c|c|c|} \hline 0 & 0 & 0 & 1 \\ \hline \end{array} \rightsquigarrow 1$$

$$\begin{array}{|c|c|c|c|} \hline 0 & 0 & 1 & 0 \\ \hline \end{array} \rightsquigarrow 2$$

■ ■ ■

$$\begin{array}{|c|c|c|c|} \hline 1 & 1 & 1 & 1 \\ \hline \end{array} \rightsquigarrow 15$$

Números enteros (int) · Operaciones

Símbolo	Operación	Ejemplo	Resultado
+	Suma	5 + 2	7
-	Resta	5 - 2	3
*	Producto	5 * 2	10
/	División	5 / 2	2.5
//	División entera	5 // 2	2
%	Resto	5 % 2	1
-	Negación	-5	-5
abs	Valor absoluto	abs(-5)	5
==	Igual	5 == 2	False
!=	Distinto	5 != 2	True
>	Mayor estricto	5 > 2	True
<	Menor estricto	5 < 2	False
>=	Mayor o igual	5 >= 2	True
<=	Menor o igual	5 <= 2	False

2.5 es de tipo float; True y False son de tipo bool.

Números enteros (int) · Ejercicio

Evaluar las siguientes expresiones en una consola `ipython`:

a) $3 + 4$

b) $3 + 2 * 2$

c) $3 + (2 * 2)$

d) $3 < 4$

e) $3 == 4 - 1$

f) $123 / 10$

g) $123 // 10$

h) $123 \% 10$

i) $123 / 0$

j) $(-1) * (-5)$

k) `abs(133)`

l) `abs(10 - 133)`

Números de punto flotante (float)

Los números de punto flotante de una computadora son **muy diferentes** de los reales matemáticos (el conjunto $\mathbb{R}$).

Está acotada su **cantidad** (son finitos) y también su **precisión**.
Por ejemplo, $\sqrt{2}$ tiene infinitos dígitos decimales (sin período):

1,41421356237309504880168872420969807856967187538...

Para representar a $\sqrt{2}$ como un número de punto flotante, las computadoras lo **truncan** en algún lugar. Por ejemplo:

1,4142135623730951

Además...

1.4142135623730951 $\rightsquigarrow$ 1.4142135623730951

1.4142135623730951234 $\rightsquigarrow$ 1.4142135623730951

1.4142135623730951412976 $\rightsquigarrow$ 1.4142135623730951

...

¡Hay **infinitos reales** representados por 1,4142135623730951!

Al trabajar con float debemos lidiar con **errores de representación**.

Números de punto flotante (float) · Operaciones

Símbolo	Operación	Ejemplo	Resultado
+	Suma	5.0 + 2.0	7.0
-	Resta	5.0 - 2.0	3.0
*	Producto	5.0 * 2.0	10.0
/	División	5.0 / 2.0	2.5
-	Negación	-5.0	-5.0
abs	Valor absoluto	abs(-5.0)	5.0
==	Igual	5.0 == 2.0	False
!=	Distinto	5.0 != 2.0	True
>	Mayor estricto	5.0 > 2.0	True
<	Menor estricto	5.0 < 2.0	False
>=	Mayor o igual	5.0 >= 2.0	True
<=	Menor o igual	5.0 <= 2.0	False

(*) No conviene comparar floats, porque los problemas de representación pueden producir resultados inesperados.

Ejemplo: 1.4142135623730951 == 1.4142135623730951234 $\rightsquigarrow$ True.

Números de punto flotante (float) · Ejercicio

Evaluar las siguientes expresiones en una consola ipython:

a) $3.0 + 4.0$

b) $3 + 4.0$

c) $1 / 3$

d) $1 / 3 == 0.33333$

e) $1 / 3 <= 0.33333$

f) $1.2 - 1.0 == 0.2$

g) $1.2 - 1.0$

h) $1.0 / 0.0$

Cadenas de caracteres (str)

Un **carácter** es un símbolo válido en la computadora:

- ▶ abcdefghijklmnopqrstuvwxyz
- ▶ ABCDEFGHIJKLMNOPQRSTUVWXYZ
- ▶ áéíóúüñÁÉÍÓÚÜÑ
- ▶ 0123456789; !#\$%*()-_+=~`' : ; , . " < > _ ? / \
- ▶ etc.

Internamente, un carácter se representa mediante un número entero.

En Python, la operación **ord** permite conocer la representación entera de un carácter, y **chr** es la operación inversa.

Ejercicio: Evaluar en una consola `ipython` las siguientes expresiones:

`ord('a')`

`chr(97)`

`chr(ord('a')+1)`

Una **cadena de caracteres** o **string** (str en Python) es una secuencia de 0 o más caracteres, delimitados por un símbolo especial (típicamente comillas simples o dobles): `'Hola, mundo!'`, `'Programming Fundamentals'`, etc.

Cadenas de caracteres (str) · Operaciones

Símbolo	Operación	Ejemplo	Resultado
<code>len(·)</code>	Longitud	<code>len('hola')</code>	4
<code>+</code>	Concatenación	<code>'algo' + 'ritmo'</code>	<code>'algoritmo'</code>
<code>*</code>	Multiplicación	<code>'ji' * 3</code>	<code>'jijiji'</code>
<code>·[·]</code>	i-ésimo carácter	<code>'aeiou'[2]</code>	<code>'i'</code>
<code>in</code>	Pertenencia	<code>'o' in 'aeiou'</code>	<code>True</code>
<code>==</code>	Igual	<code>'hola' == 'mundo'</code>	<code>False</code>
<code>!=</code>	Distinto	<code>'hola' != 'mundo'</code>	<code>True</code>
<code>></code>	Mayor estricto	<code>'hola' > 'mundo'</code>	<code>False</code>
<code><</code>	Menor estricto	<code>'hola' < 'mundo'</code>	<code>True</code>
<code>>=</code>	Mayor o igual	<code>'hola' >= 'mundo'</code>	<code>False</code>
<code><=</code>	Menor o igual	<code>'hola' <= 'mundo'</code>	<code>True</code>

Las comparaciones por mayor y menor se basan en las representaciones enteras de los caracteres (se compara carácter a carácter, de izquierda a derecha).

Hay muchísimas otras operaciones con strings; por ahora con estas nos alcanza.

Cadenas de caracteres (str) · Ejercicio

Evaluar las siguientes expresiones en una consola ipython:

- a) `'prog' + 'rama'`
- b) `'3' + '4'`
- c) `'ta' * 5`
- d) `'universidad'[3]`
- e) `'ver' in 'universidad'`
- f) `'VER' in 'universidad'`
- g) `'a' < 'b'`
- h) `'a' < 'B'`

¿Por qué las últimas dos expresiones devuelven resultados distintos?

Expresiones

Una **expresión** es una combinación de valores, variables y operadores (incluyendo llamados a funciones).

La **evaluación** de una expresión arroja como resultado un valor.

Ejercicio: ¿Qué valores resultan de evaluar estas expresiones, y cuál es el tipo de cada una?

a) $2 + 3 * 4$

b) $1 / 2 + 5.0$

c) $'2' + 'dos' * 2$

Primero **pensar** las respuestas. Después evaluarlas en una consola `ipython` y averiguar sus tipos usando la operación `type()`.

Prestar atención a la **precedencia de los operadores**, que determina su orden de evaluación.

Sobrecarga de operadores

Se dice que operadores como `+` o `==` están **sobrecargados**: denotan operaciones en más de un tipo de datos. Se desambiguan por el contexto:

- ▶ `1 + 2` $\rightsquigarrow$ `3` (`int`)
- ▶ `1.0 + 2.0` $\rightsquigarrow$ `3.0` (`float`)
- ▶ `'1' + '2'` $\rightsquigarrow$ `'12'` (`str`)

Expresiones potencialmente ambiguas como `1 + '2'` o `1 == '2'` pueden arrojar un **error de tipos** o resultados inesperados.

Conversión de tipos

A veces tiene sentido convertir una expresión de cierto tipo a otro tipo.

Ejemplos:

- ▶ `int('1234')` convierte el string '1234' en el entero 1234.
- ▶ `float(5)` convierte el entero 5 en el número de punto flotante 5.0.
- ▶ `str(9876)` convierte el entero 9876 en el string '9876'.
- ▶ `int(3.1416)` convierte el float 3.1416 en el entero 3. En este caso, decimos que la parte decimal del número *se trunca*.

Cuando la conversión no tenga sentido, obtendremos un error.

Ejemplo:

- ▶ `int('x') → ValueError: invalid literal for int() [...]`

Ejercicio: (tarea) Averiguar el tipo de la siguiente expresión y evaluarla:

```
str(int(float('123.45')))[1] * 3
```

Resolverlo primero a mano, y después revisar en una consola `ipython`.

Repaso de la clase de hoy

- ▶ Tipos de datos: `int`, `float`, `str`.
- ▶ ¡Cuidado con las representaciones de los números en la computadora! En especial, cuidado con los errores numéricos del tipo `float`.
- ▶ Evaluación de expresiones; sobrecarga de operadores; errores de tipos; conversión de tipos.

Bibliografía complementaria:

- ▶ IPP, secciones 3.1, 3.5 a 3.10 (ignorar variables por ahora), 4.1 y 4.2.

Con lo visto, ya pueden resolver hasta el ejercicio 1.4 de la sección 1 de la guía de ejercicios.